The reasoning behind these restrictions goes back to one of the guidelines from chapter 3: don't pass pointers and references to protected data outside the scope of the lock by passing them as arguments to user-supplied functions. In general, the compiler isn't going to be able to generate lock-free code for `std::atomic<UDT>`, so it will have to use an internal lock for all the operations. If user-supplied copy-assignment or comparison operators were permitted, this would require passing a reference to the protected data as an argument to a user-supplied function, thus violating the guideline. Also, the library is entirely at liberty to use a single lock for all atomic operations that need it, and allowing user-supplied functions to be called while holding that lock might cause deadlock or cause other threads to block because a comparison operation took a long time. Finally, these restrictions increase the chance that the compiler will be able to make use of atomic instructions directly for `std::atomic<UDT>` (and thus make a particular instantiation lock-free), because it can just treat the user-defined type as a set of raw bytes.

Note that although you can use `std::atomic<float>` or `std::atomic<double>`, because the built-in floating point types do satisfy the criteria for use with `memcpy` and `memcmp`, the behavior may be surprising in the case of `compare_exchange_strong`. The operation may fail even though the old stored value was equal in value to the comparand, if the stored value had a different representation. Note that there are no atomic arithmetic operations on floating-point values. You'll get similar behavior with `compare_exchange_strong` if you use `std::atomic<>` with a user-defined type that has an equality-comparison operator defined, and that operator differs from the comparison using `memcmp`—the operation may fail because the otherwise-equal values have a different representation.

If your `UDT` is the same size as (or smaller than) an `int` or a `void*`, most common platforms will be able to use atomic instructions for `std::atomic<UDT>`. Some platforms will also be able to use atomic instructions for user-defined types that are twice the size of an `int` or `void*`. These platforms are typically those that support a so-called *double-word-compare-and-swap (DWCAS)* instruction corresponding to the `compare_exchange_xxx` functions. As you'll see in chapter 7, such support can be helpful when writing lock-free code.

These restrictions mean that you can't, for example, create a `std::atomic<std::vector<int>>`, but you can use it with classes containing counters or flags or pointers or even just arrays of simple data elements. This isn't particularly a problem; the more complex the data structure, the more likely you'll want to do operations on it other than simple assignment and comparison. If that's the case, you're better off using a `std::mutex` to ensure that the data is appropriately protected for the desired operations, as described in chapter 3.

When instantiated with a user-defined type `T`, the interface of `std::atomic<T>` is limited to the set of operations available for `std::atomic<bool>`: `load()`, `store()`, `exchange()`, `compare_exchange_weak()`, `compare_exchange_strong()`, and assignment from and conversion to an instance of type `T`.

Table 5.3 shows the operations available on each atomic type.

5.2.7. Free functions for atomic operations